

JOB RECOMMENDATION SYSTEM

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: df = pd.read_csv("Combined_Jobs_Final.csv.zip", compression="zip")
```

```
In [3]: df.head(5)
```

Out[3]:

	Job.ID	Provider	Status	Slug	Title	Position	Company	City	State.Name	State.Code	...	
0	111	1	open	palo-alto-ca-tacolicious-server	Server @ Tacolicious	Server	Tacolicious	Palo Alto	California	CA	...	E
1	113	1	open	san-francisco-ca-claudelane-kitchen-staff-chef	Kitchen Staff/Chef @ Claude Lane	Kitchen Staff/Chef	Claude Lane	San Francisco	California	CA	...	E
2	117	1	open	san-francisco-ca-machka-restaurants-corp-barte...	Bartender @ Machka Restaurants Corp.	Bartender	Machka Restaurants Corp.	San Francisco	California	CA	...	E
3	121	1	open	brisbane-ca-teriyaki-house-server	Server @ Teriyaki House	Server	Teriyaki House	Brisbane	California	CA	...	E
4	127	1	open	los-angeles-ca-rosa-mexicano-sunset-kitchen-st...	Kitchen Staff/Chef @ Rosa Mexicano - Sunset	Kitchen Staff/Chef	Rosa Mexicano - Sunset	Los Angeles	California	CA	...	E

5 rows × 23 columns

INFORMATION

```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 84090 entries, 0 to 84089
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Job.ID                 84090 non-null  int64
1   Provider               84090 non-null  int64
2   Status                 84090 non-null  object
3   Slug                   84090 non-null  object
4   Title                  84090 non-null  object
5   Position               84090 non-null  object
6   Company                81819 non-null  object
7   City                   83955 non-null  object
8   State.Name             83919 non-null  object
9   State.Code             83919 non-null  object
10  Address                 36 non-null     object
11  Latitude                84090 non-null  float64
12  Longitude               84090 non-null  float64
13  Industry                267 non-null    object
14  Job.Description         84034 non-null  object
15  Requirements            0 non-null      float64
16  Salary                  229 non-null    float64
17  Listing.Start           83407 non-null  object
18  Listing.End             83923 non-null  object
19  Employment.Type         84080 non-null  object
20  Education.Required      83823 non-null  object
21  Created.At              84090 non-null  object
22  Updated.At              84090 non-null  object
dtypes: float64(4), int64(2), object(17)
memory usage: 14.8+ MB
```

```
In [5]: df.shape
```

```
Out[5]: (84090, 23)
```

```
In [7]: df.isnull().sum()
df.fillna('',inplace=True)
df.isnull().sum()
```

C:\Users\Komal\AppData\Local\Temp\ipykernel_51608\3427377440.py:2: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value '' has dtype incompatible with float64, please explicitly cast to a compatible dtype first.

```
df.fillna('',inplace=True)
```

```
Out[7]: Job.ID                0
Provider                0
Status                  0
Slug                    0
Title                   0
Position                0
Company                 0
City                    0
State.Name              0
State.Code              0
Address                 0
Latitude                0
Longitude               0
Industry                0
Job.Description         0
Requirements            0
Salary                  0
Listing.Start           0
Listing.End             0
Employment.Type         0
Education.Required      0
Created.At              0
Updated.At              0
dtype: int64
```

```
In [10]: df = df.sample(n=2000,random_state=42)
```

```
In [12]: df.shape
```

```
Out[12]: (2000, 23)
```

```
In [13]: import nltk
import re
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
ps = PorterStemmer()
```

```
In [14]: def cleaning(txt):
txt = re.sub(r'^a-zA-Z0-9\s','',txt)
```

```
tokens = nltk.word_tokenize(txt.lower())
stemming = [ps.stem(w) for w in tokens if w not in stopwords.words('english')]
return " ".join(stemming)
```

```
In [15]: cleaning(" this is a book #$$$")
```

```
Out[15]: 'book'
```

```
In [17]: df['Job.Description'] = df['Job.Description'].astype(str).apply(lambda x: cleaning(x))
df['Title'] = df['Title'].astype(str).apply(lambda x: cleaning(x))
```

```
In [18]: df['clean_text'] = df['Job.Description']+" "+df['Title']
```

Vectorization

```
In [19]: from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
In [20]: tfidf = TfidfVectorizer(stop_words='english')
matrix = tfidf.fit_transform(df['clean_text'])
similarity = cosine_similarity(matrix)
```

```
In [21]: sorted(list(enumerate(similarity[0])), key=lambda x: x[1], reverse=True)[1:50]
```

```
Out[21]: [(276, np.float64(0.9939300864743976)),
(730, np.float64(0.532157408787013)),
(1019, np.float64(0.5139380184793445)),
(81, np.float64(0.5131738747262427)),
(917, np.float64(0.5131738747262427)),
(1081, np.float64(0.5131738747262427)),
(1454, np.float64(0.5131738747262427)),
(252, np.float64(0.27242408742083685)),
(128, np.float64(0.23723817319589838)),
(360, np.float64(0.2308000230388527)),
(1465, np.float64(0.21168666990788051)),
(825, np.float64(0.20947679993040488)),
(1715, np.float64(0.20718765642631232)),
(1113, np.float64(0.2070838625127794)),
(1989, np.float64(0.20526729919394623)),
(629, np.float64(0.19842678388659027)),
(245, np.float64(0.18682812566676812)),
(38, np.float64(0.17885677785062074)),
(1602, np.float64(0.16963330922232517)),
(254, np.float64(0.16862005255488166)),
(1829, np.float64(0.1662338648346323)),
(1744, np.float64(0.16284557359881457)),
(1501, np.float64(0.15682168543475247)),
(114, np.float64(0.15463991428916557)),
(59, np.float64(0.15151421257118763)),
(298, np.float64(0.15045488542600766)),
(1711, np.float64(0.15045488542600766)),
(469, np.float64(0.14973759724382124)),
(965, np.float64(0.1456040930819451)),
(319, np.float64(0.14553146023755517)),
(1026, np.float64(0.14316461266138417)),
(655, np.float64(0.1418280555616395)),
(284, np.float64(0.13899843403861814)),
(195, np.float64(0.13894384636905172)),
(1528, np.float64(0.13523482056954614)),
(539, np.float64(0.13490050790061653)),
(326, np.float64(0.13352539143723904)),
(528, np.float64(0.13304990363128558)),
(665, np.float64(0.1327359339007512)),
(113, np.float64(0.13251103436571413)),
(1338, np.float64(0.13251103436571413)),
(940, np.float64(0.1323572129359167)),
(433, np.float64(0.12774070559923054)),
(1686, np.float64(0.1255815630196375)),
(857, np.float64(0.12472225156521731)),
(52, np.float64(0.12390461395628863)),
(658, np.float64(0.12390461395628863)),
(1273, np.float64(0.12390461395628863)),
(398, np.float64(0.12263670164383848))]
```

Recommendation

```
In [22]: def recommend(title):
        indx = df[job_df['Title'] == title].index[0]
```

```
indx = df.index.get_loc(indx)
distances = sorted(list(enumerate(similarity[indx])), key=lambda x: x[1], reverse=True)[1:50])
jobs = []
for i in distances:
    jobs.append(df.iloc[i[0]].Title)
return jobs
```

```
In [25]: import pickle
pickle.dump(df, open('jk.pkl', 'wb'))
pickle.dump(similarity, open('similarity.pkl', 'wb'))
```

```
In [ ]:
```